

CS707 – Generative Models for NLP and Multimodal Reasoning

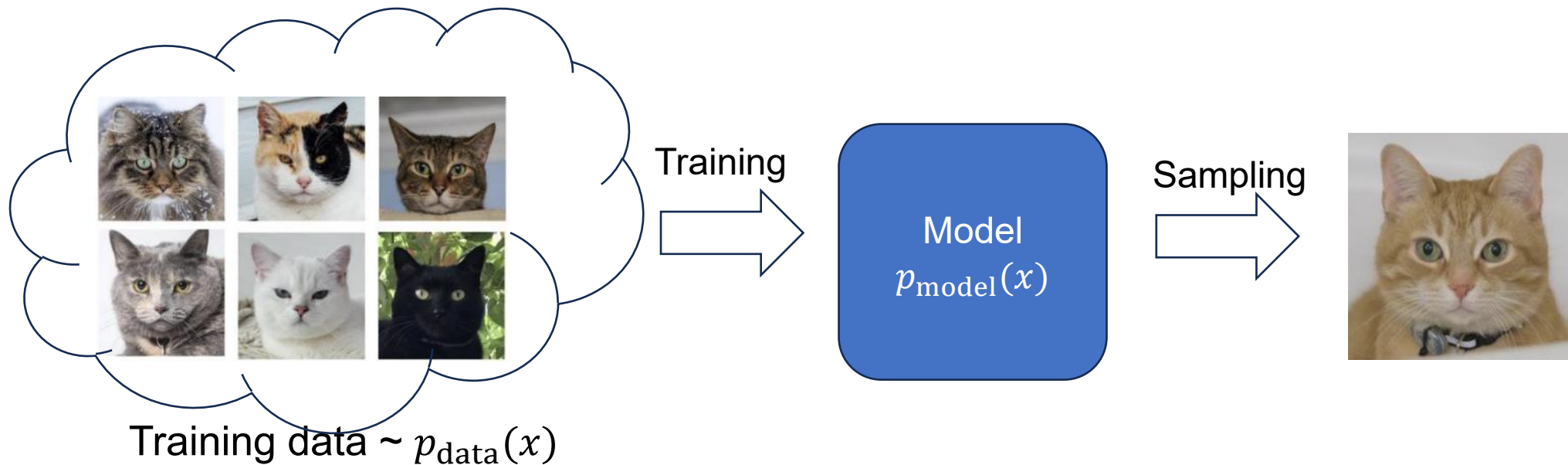
2026-27 Term 1

Week 2: Probabilistic and Neural Generative Models

Dr. Wei Gao
Office: SCIS-1/4057
Email: weigao@smu.edu.sg
Tel.: 68261369

Generative Modeling

Given training data, generate new samples from same distribution

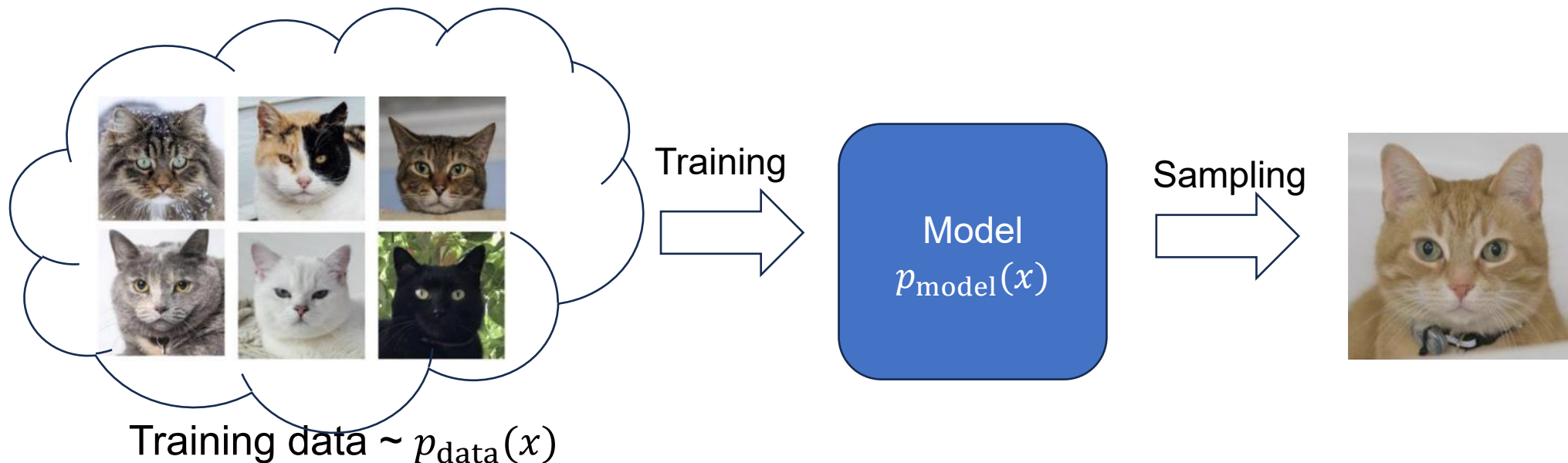


Objectives:

1. Learn $p_{\text{model}}(x)$ that approximate $p_{\text{data}}(x)$
2. **Sampling new x from $p_{\text{model}}(x)$**

Generative Modeling

Given training data, generate new samples from same distribution



Formulate as density estimation problems:

- **Explicit density estimation:** explicitly define and solve for $p_{\text{model}}(x)$
- **Implicit density estimation:** learn model that can sample from $p_{\text{model}}(x)$ without explicitly defining it.

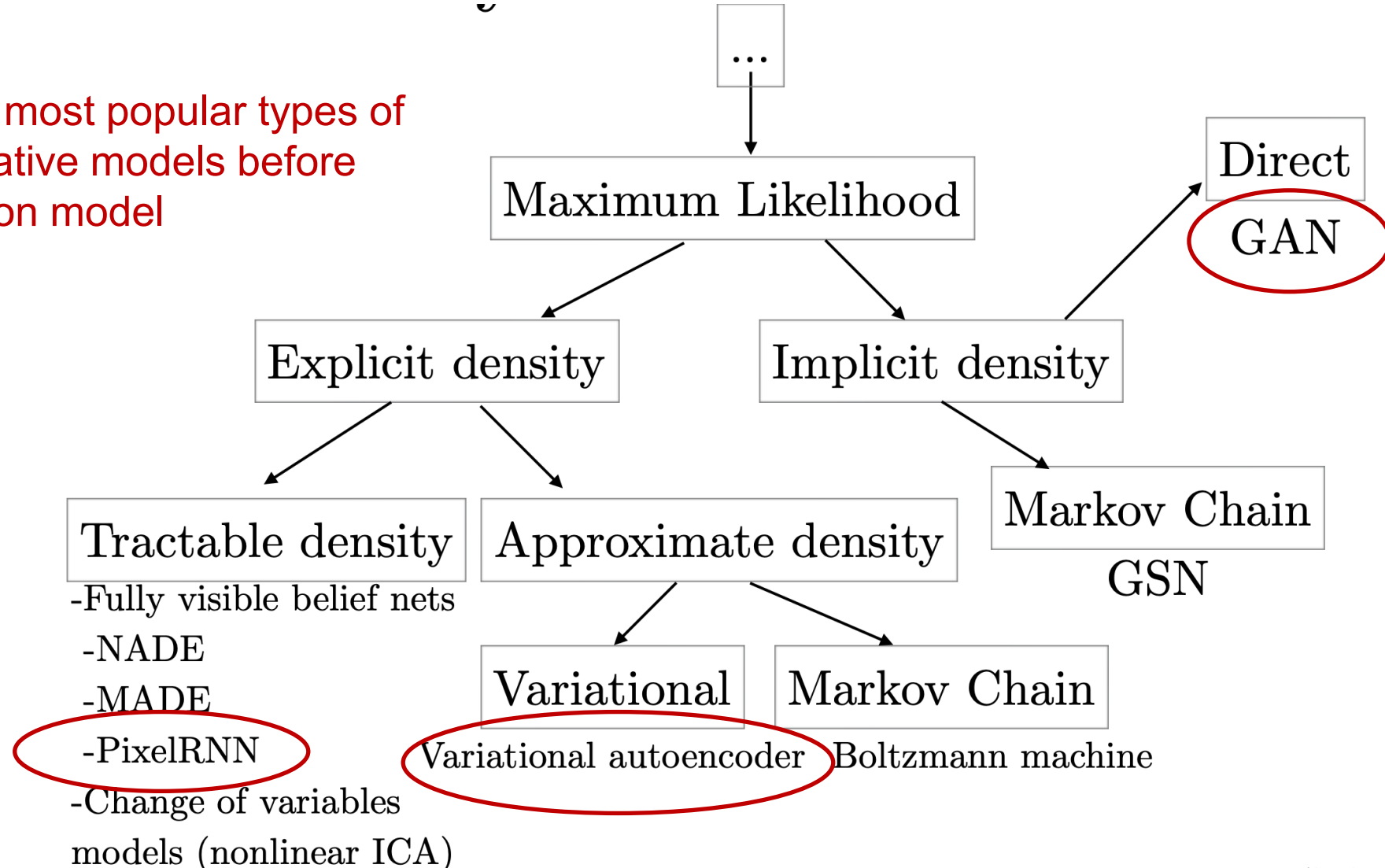
Why Generative Models?



- Realistic samples for artwork, super-resolution, colorization, etc.
- Learn useful features for downstream tasks such as classification.
- Getting insights from high-dimensional data (physics, medical imaging, etc.)
- Modeling physical world for simulation and planning (robotics and reinforcement learning applications)
- ...

Taxonomy of Generative Models (Goodfellow 2016)

Three most popular types of generative models before diffusion model



Outline

- PixelRNN/PixelCNN
- VAE
- GAN
- Diffusion (Week 9)

Fully visible belief network (FVBN)

Explicit density model

Use chain rule to decompose likelihood of an image x into product of 1-d distributions:

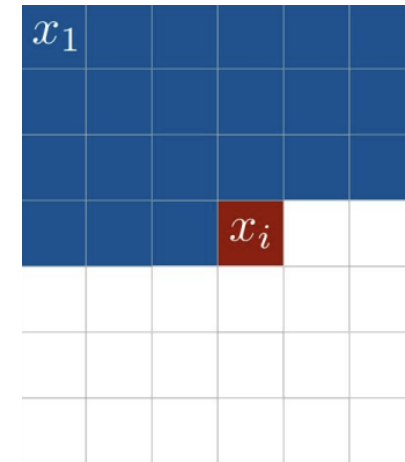
$$p(x) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$$

↑

Likelihood of
image x

↑

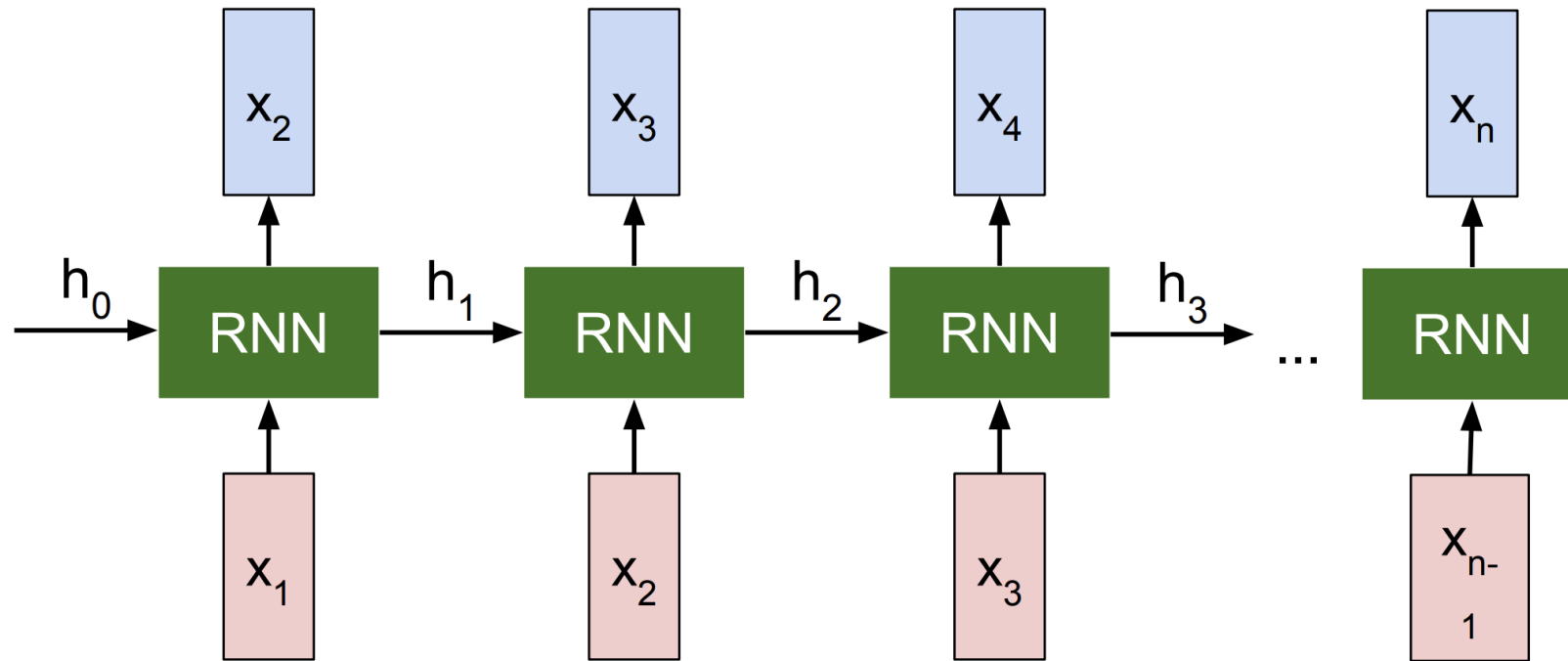
Probability of i -th pixel value
given all previous pixels



Then maximize likelihood of training data

Complex distribution over pixel values => Express using a neural network!

Recurrent Neural Network



$$p(x_i | x_1, \dots, x_{i-1})$$

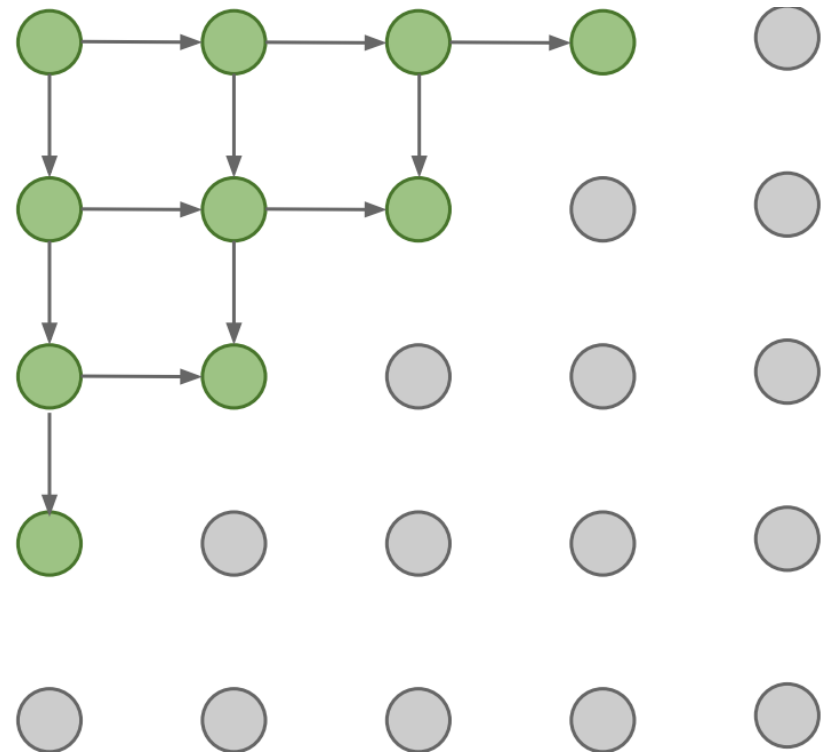
PixelRNN (van der Oord et al., 2016)

Generate image pixels starting from corner

Dependency on previous pixels modeled using an RNN (LSTM)

$$p(x) = \prod_{t=1}^{HW \cdot C} p(x_t \mid x_{<t})$$

Drawback: sequential generation is slow in both training and inference!



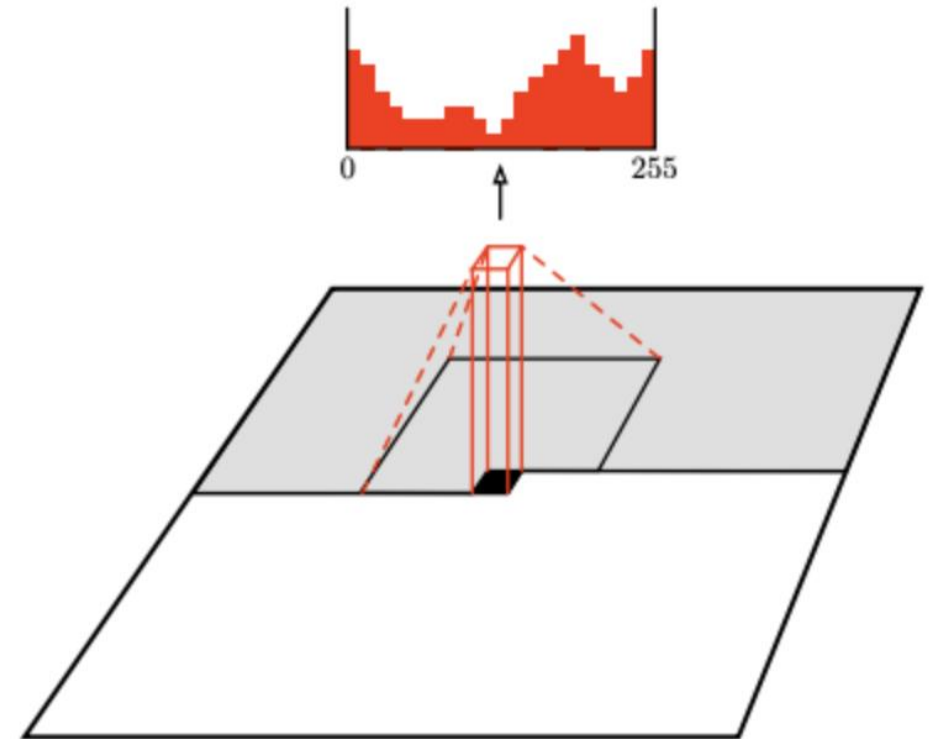
PixelCNN (van der Oord et al., 2016)

Still generate image pixels starting from corner

Dependency on previous pixels now modeled using a CNN over context region
(masked convolution)

Training is faster than PixelRNN
(can parallelize convolutions since context region values known from training images)

Generation is still slow:
For a 32x32 image, we need to do forward passes of the network 1024 times for a single image



Generation Samples

Samples from a model trained on ImageNet 32x32 images.



PixelRNN and PixelCNN

Pros:

- Can explicitly compute likelihood $p(x)$
- Easy to optimize
- Produce good samples

Cons:

- Sequential generation => slow

Improving PixelCNN performance

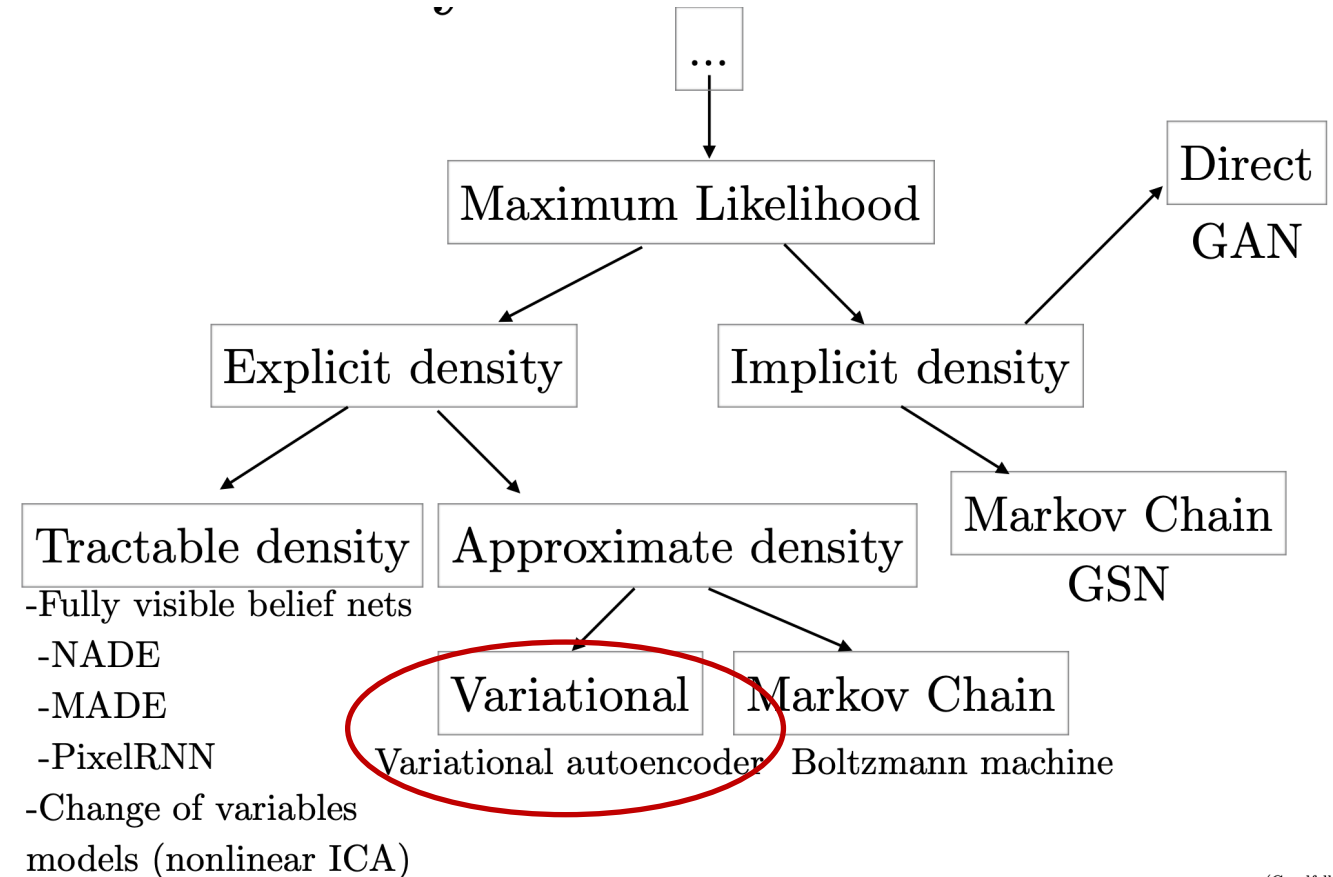
- Gated convolutional layers
- Short-cut connections
- Discretized logistic loss
- Multi-scale
- Training tricks
- Etc...

See

- Van der Oord et al. NIPS 2016
- Salimans et al. 2017 (PixelCNN++)

Outline

- PixelRNN/PixelCNN
- VAE
- GAN
- Diffusion (Week 9)



So far...

PixelRNN/PixelCNN define tractable density function, optimize likelihood of training data:

$$p_{\theta}(x) = \prod_{i=1}^n p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

Variational Autoencoders (VAEs) define intractable density function with latent $\mathbf{z}$:

$$p_{\theta}(x) = \int p_{\theta}(z) \prod_{i=1}^n p_{\theta}(x_i | z) dz$$

No dependencies among pixels, can generate all pixels at the same time!

Cannot optimize directly, derive and optimize lower bound on likelihood instead

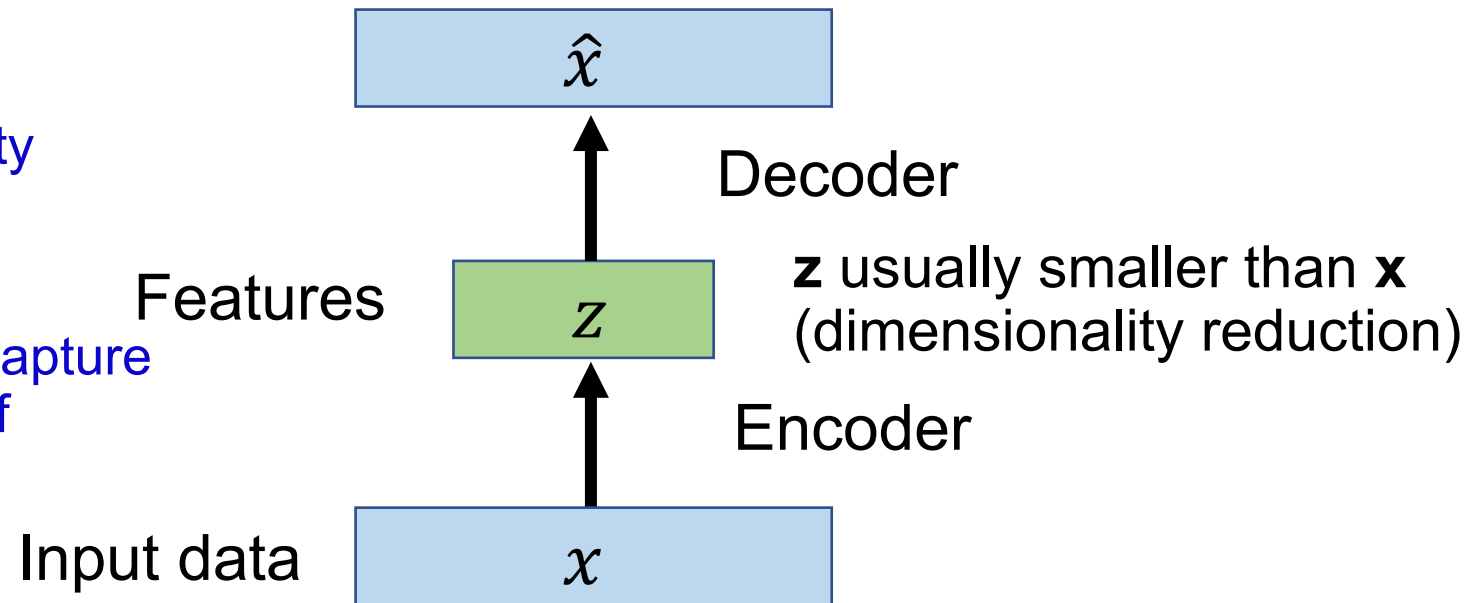
Why latent z ?

Some background: Autoencoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training

Q: why dimensionality reduction?

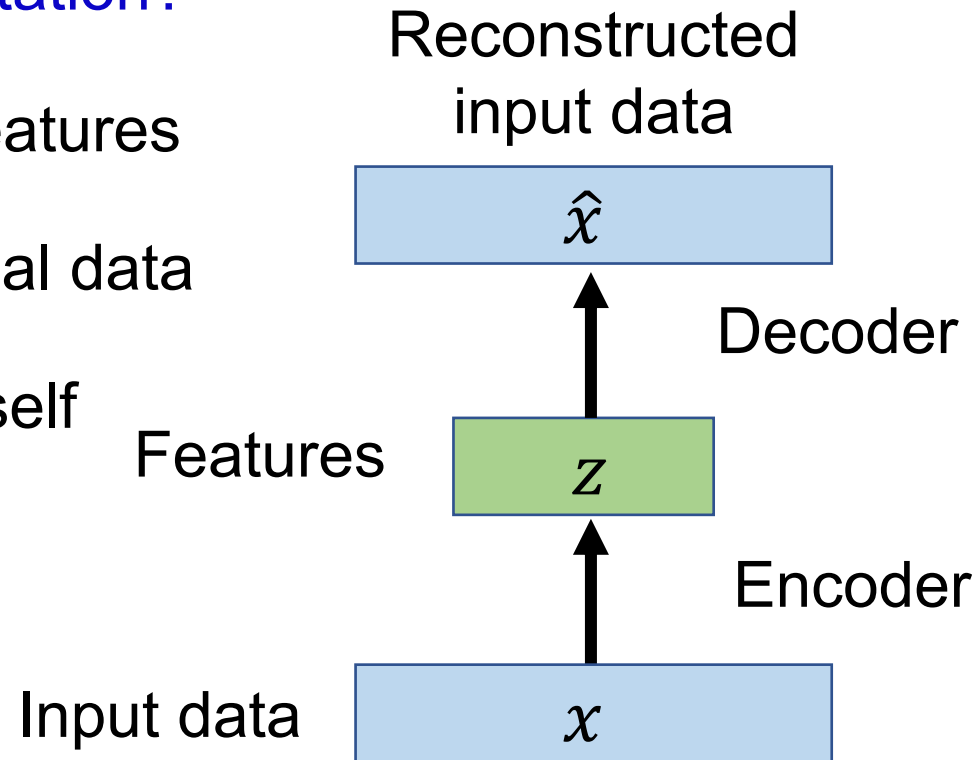
A: want features to capture meaningful factors of variation in data



Some background: Autoencoders

How to learn this feature representation?

Train such that features can be used to reconstruct original data
“Autoencoding” - encoding input itself



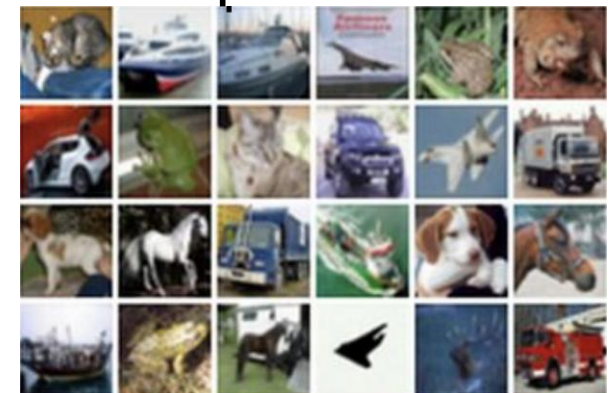
Reconstructed data



Decoder: 4-layer upconv

Encoder: 4-layer conv

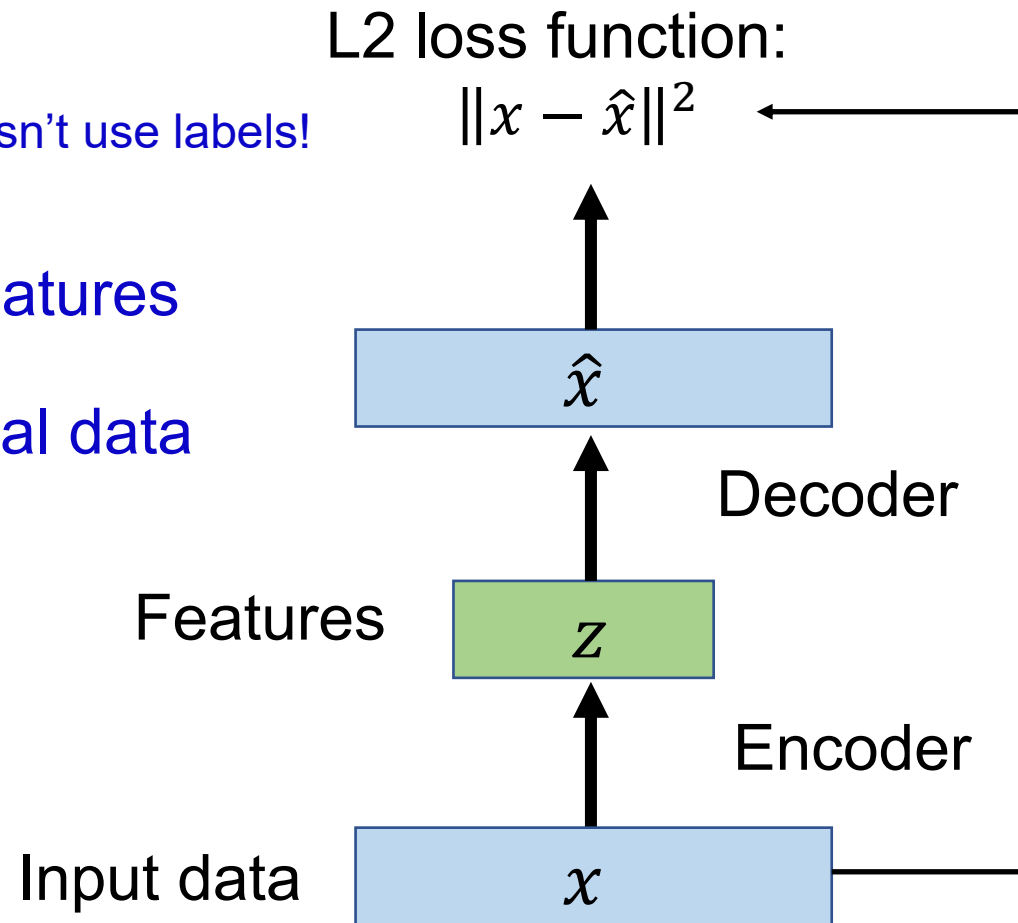
Input data



Some background: Autoencoders

Train such that features can be used to reconstruct original data

Doesn't use labels!

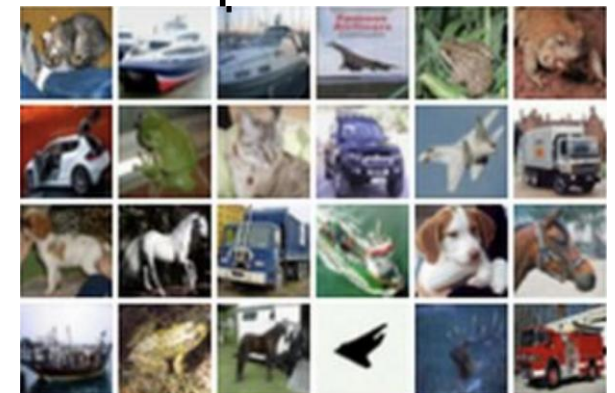


Reconstructed data

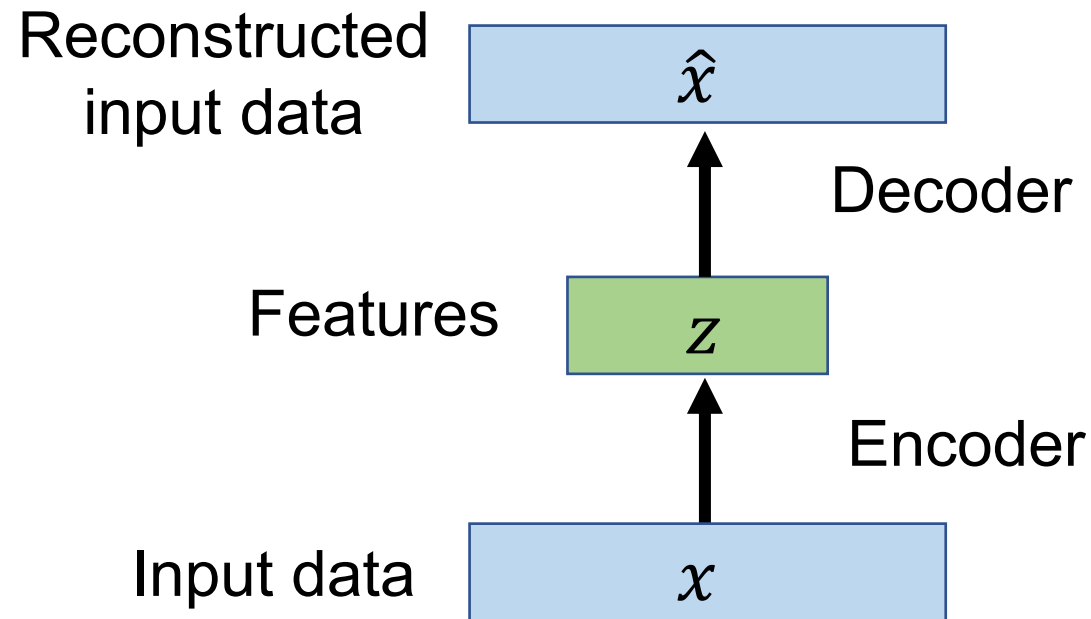


Encoder: 4-layer conv
Decoder: 4-layer upconv

Input data



Some background: Autoencoders

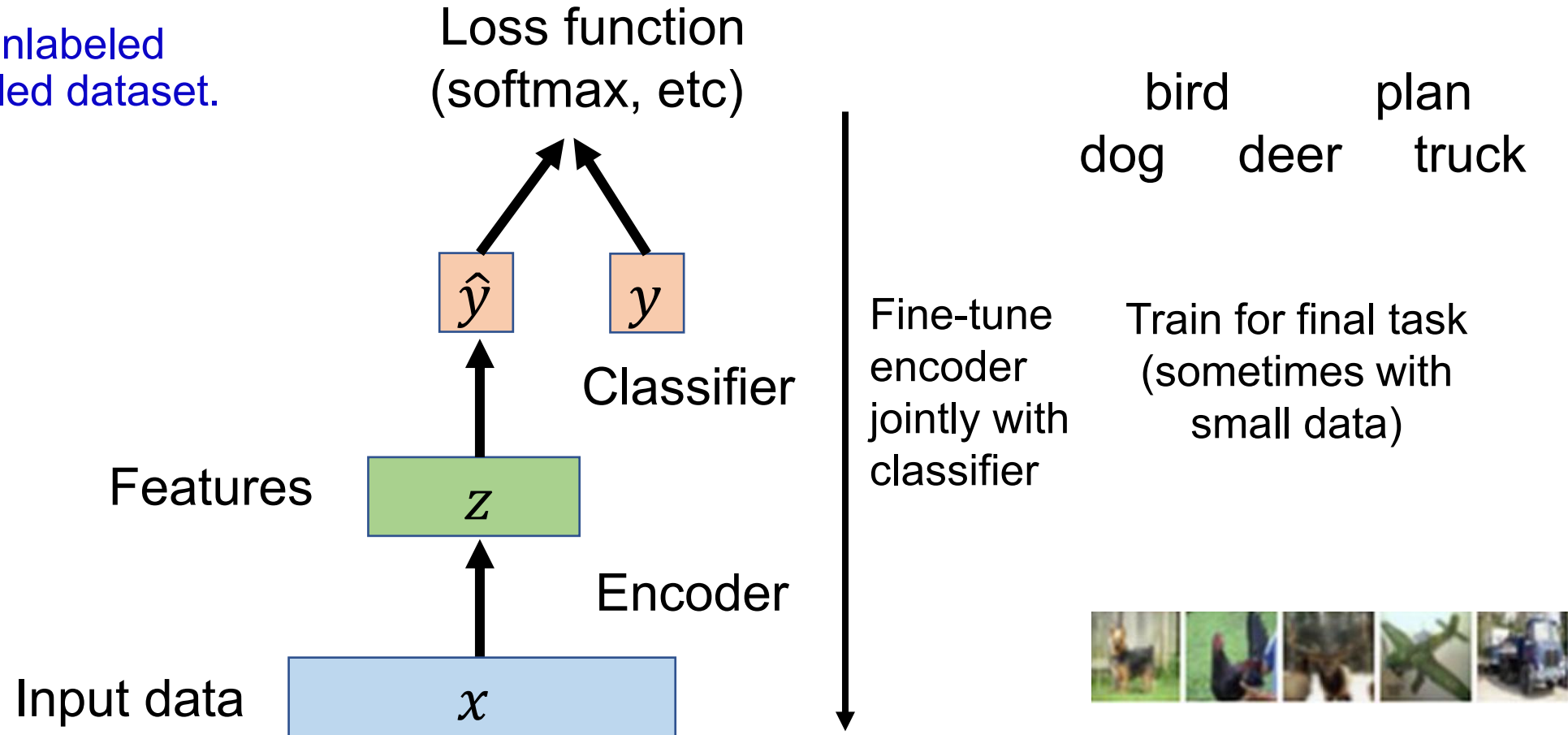


After training, throw away decoder

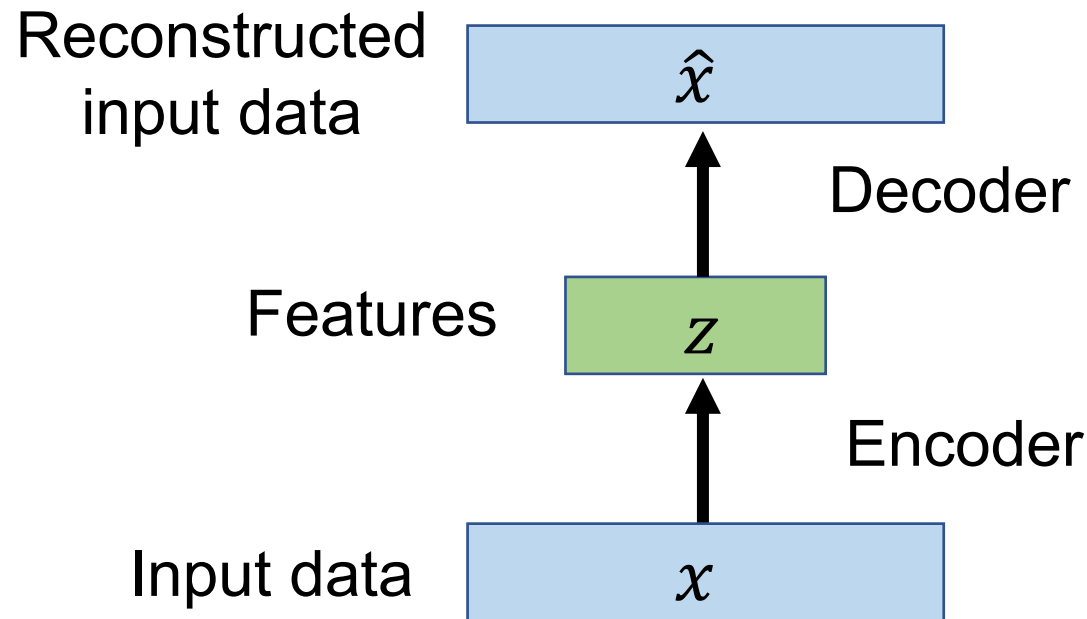
Some background: Autoencoders

Transfer from large, unlabeled dataset to small, labeled dataset.

Encoder can be used to initialize a **supervised** model



Some background: Autoencoders



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data.

But we can't generate new images from an autoencoder because we don't know the space of z .

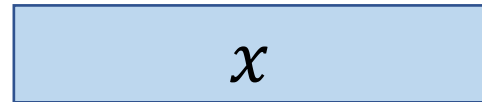
How do we make autoencoder a **generative model**?

Variational Autoencoders

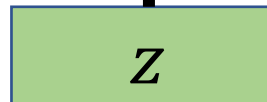
Probabilistic spin on autoencoders – will let us sample from the model to generate data!

Assume training data $\{x\}$ is generated from the **distribution** of unobserved (latent) representation z

Sample from
true conditional
 $p_{\theta^*}(x|z^{(i)})$



Sample from
true prior
 $z^{(i)} \sim p_{\theta^*}(z)$



Intuition (remember from autoencoders!): x is an image, z is latent factors used to generate x : attributes, orientation, etc

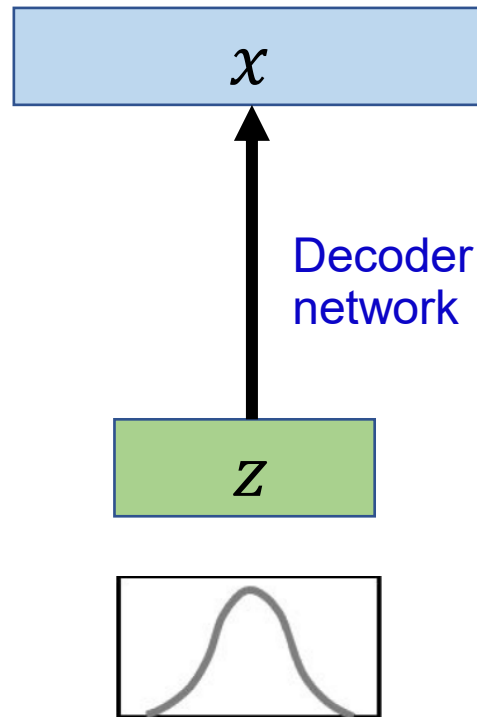
Variational Autoencoders



We want to estimate the true parameters θ^* of this generative model given training data $\{x\}$.

Sample from true conditional $p_{\theta^*}(x|z^{(i)})$

Sample from true prior $z^{(i)} \sim p_{\theta^*}(z)$



How should we represent this model?

Choose prior $p(z)$ to be simple, e.g. Gaussian. Reasonable for latent attributes, e.g., pose, how much smile.

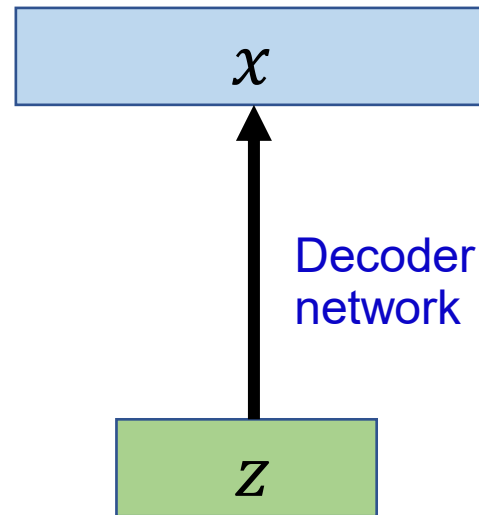
Conditional $p(x|z)$ is complex (generates image) => **represent with neural network**

Variational Autoencoders

We want to estimate the true parameters θ^* of this generative model given training data $\{x\}$.

Sample from true conditional $p_{\theta^*}(x|z^{(i)})$

Sample from true prior $z^{(i)} \sim p_{\theta^*}(z)$



How to train the model?

Learn model parameters to maximize likelihood of training data.

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

Q: What is the problem with this?
Intractable!

Variational Autoencoders: Intractability

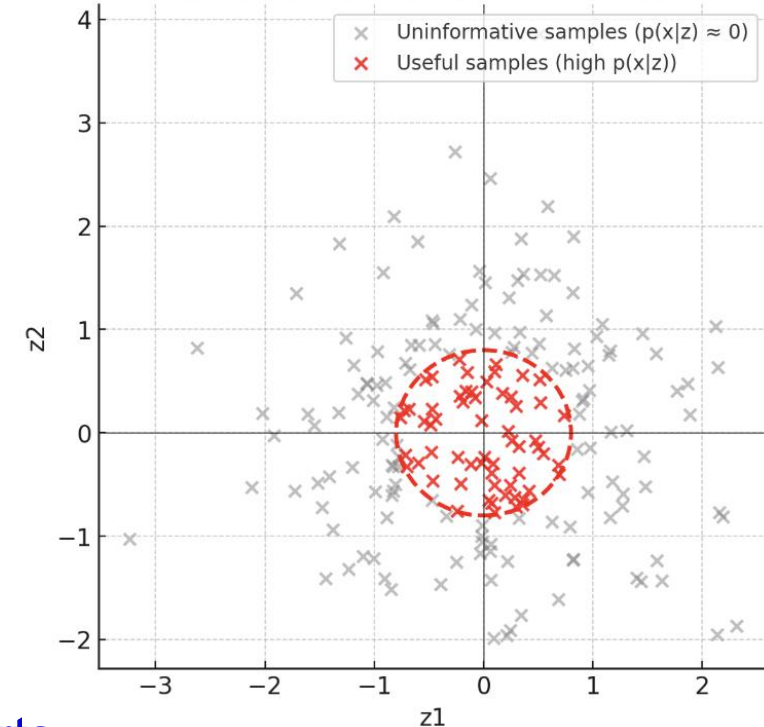
Data likelihood: $p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$

Simple Gaussian prior network
Intractable to compute $p(x|z)$ for every z !

$$\log p(x) \approx \log \frac{1}{k} \sum_{i=1}^k p(x|z^{(i)}), \text{ where } z^{(i)} \sim p(z)$$

Monte Carlo estimation is too high variance

Monte Carlo Sampling in Latent Space (Throwing Darts Analogy)



This motivates **variational inference**: instead of throwing darts blindly from $p(z)$, learn a proposal distribution $q(z | x)$ approximating $p(z | x)$ that aims darts closer to the bullseye for each x .

Variational Autoencoders: Intractability

Data likelihood: $p_{\theta}(x) = \int \overset{\checkmark}{p_{\theta}(z)} \overset{\checkmark}{p_{\theta}(x|z)} dz$

Posterior density: ~~also intractable~~ $p_{\theta}(z|x) = p_{\theta}(z)p_{\theta}(x|z)/p_{\theta}(x)$

↑
Intractable data likelihood

Solution: In addition to modeling $p_{\theta}(x|z)$, learn $q_{\phi}(z|x)$ that approximates the true posterior $p_{\theta}(z|x)$.

Will see that the approximate posterior allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize.

Variational inference is to approximate the unknown posterior distribution from only the observed data x

Variational Autoencoders

$$\log p_{\theta}(x^{(i)}) = \mathbf{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})]$$

$p_{\theta}(x^{(i)})$ does not depend on z

$$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right]$$

Bayes' Rule

Taking expectation wrt. z
(using encoder network) will
come in handy later

$$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \frac{q_{\phi}(z|x^{(i)})}{q_{\phi}(z|x^{(i)})} \right]$$

Multiply by constant

$$= \mathbf{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbf{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbf{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right]$$

Logarithms

$$= \mathbf{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbf{D}_{KL}(q_{\phi}(z|x^{(i)}) || p_{\theta}(z)) + \mathbf{D}_{KL}(q_{\phi}(z|x^{(i)}) || p_{\theta}(z|x^{(i)}))$$

The expectation wrt. z
(using encoder network) let
us write nice KL terms

Variational Autoencoders

$\log p_{\theta}(x^{(i)}) = \mathbf{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})]$
 $p_{\theta}(x^{(i)})$ does not depend on z

$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right]$
 Bayes' Rule

$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \frac{q_{\phi}(z|x^{(i)})}{q_{\phi}(z|x^{(i)})} \right]$
 Multiply by constant

$= \mathbf{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbf{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbf{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right]$
 Logarithms

$= \mathbf{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbf{D}_{KL}(q_{\phi}(z|x^{(i)}) || p_{\theta}(z)) + \mathbf{D}_{KL}(q_{\phi}(z|x^{(i)}) || p_{\theta}(z|x^{(i)}))$

We want to
maximize the
data
likelihood

Decoder network gives $p_{\theta}(x|z)$, can
compute estimate of this term through
sampling (need some trick to
differentiate through sampling).

This KL term (between
Gaussians for encoder and z
prior) has nice closed-form
solution!

$p_{\theta}(z|x)$ intractable (saw
earlier), can't compute this KL
term :(But we know KL
divergence always ≥ 0 .

Variational Autoencoders

$\log p_{\theta}(x^{(i)}) = \mathbf{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})]$
 $p_{\theta}(x^{(i)})$ does not depend on z


 We want to maximize the data likelihood

$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right]$
 Bayes' Rule

$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \frac{q_{\phi}(z|x^{(i)})}{q_{\phi}(z|x^{(i)})} \right]$
 Multiply by constant

$= \mathbf{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbf{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbf{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right]$
 Logarithms

$= \underbrace{\mathbf{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbf{D}_{KL}(q_{\phi}(z|x^{(i)}) || p_{\theta}(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)} + \underbrace{\mathbf{D}_{KL}(q_{\phi}(z|x^{(i)}) || p_{\theta}(z|x^{(i)}))}_{\geq 0}$

ELBO: Tractable lower bound which we can take gradient of and optimize! ($p_{\theta}(x|z)$ differentiable, KL term differentiable)

Variational Autoencoders

$$\log p_{\theta}(x^{(i)}) = \mathbf{E}_{z \sim q_{\phi}(z|x^{(i)})} [\log p_{\theta}(x^{(i)})]$$

$p_{\theta}(x^{(i)})$ does not depend on z

$$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \right]$$

Bayes' Rule

$$= \mathbf{E}_z \left[\log \frac{p_{\theta}(x^{(i)}|z)p_{\theta}(z)}{p_{\theta}(z|x^{(i)})} \frac{q_{\phi}(z|x^{(i)})}{q_{\phi}(z|x^{(i)})} \right]$$

Multiply by constant

$$= \mathbf{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbf{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z)} \right] + \mathbf{E}_z \left[\log \frac{q_{\phi}(z|x^{(i)})}{p_{\theta}(z|x^{(i)})} \right]$$

Encoder:
make approximate
posterior distribution
close to prior

Logarithms

$$\geq \underbrace{\mathbf{E}_z [\log p_{\theta}(x^{(i)}|z)] - \mathbf{D}_{KL}(q_{\phi}(z|x^{(i)}) || p_{\theta}(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

ELBO: Tractable lower bound which we can take gradient of and optimize! ($p_{\theta}(x|z)$ differentiable, KL term differentiable)

Decoder:
reconstruct
the input data

Variational Autoencoders

Putting it all together: maximizing the likelihood lower bound

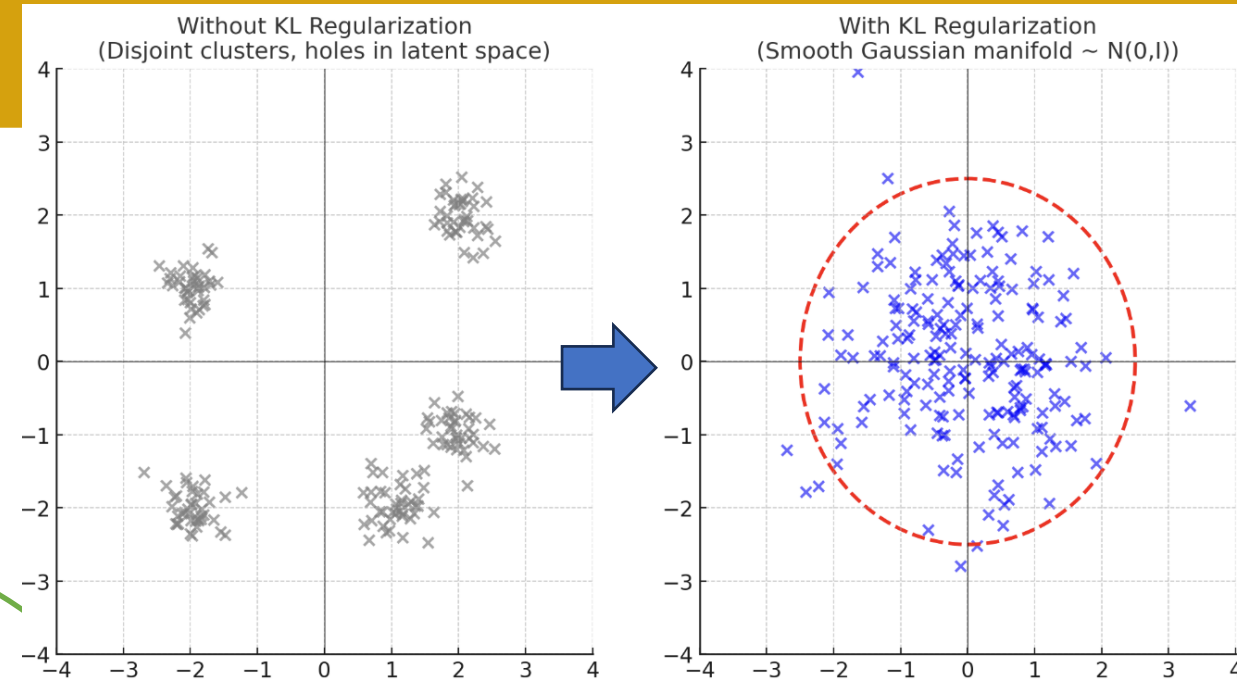
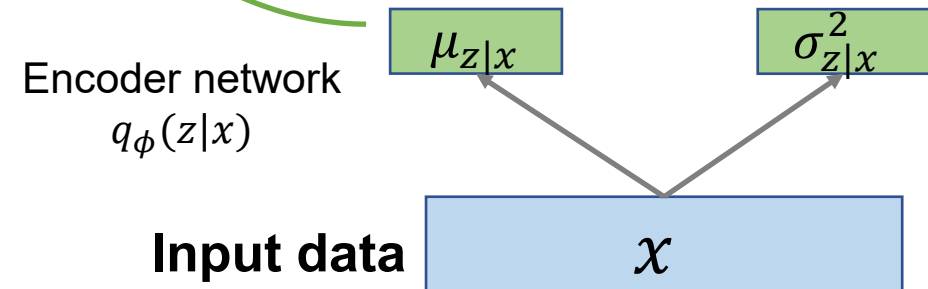
$$\underbrace{\mathbb{E}_z[\log p_\theta(x^{(i)}|z)] - \mathbf{D}_{KL}(q_\phi(z|x^{(i)})||p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Let's look at computing the KL divergence between the estimated posterior and the prior given some data

Make approximate posterior distribution close to prior

$$\mathbf{D}_{KL}(\mathcal{N}(\mu_{z|x}, \sigma_{z|x}^2) || \mathcal{N}(0, I))$$

Have a closed-form solution

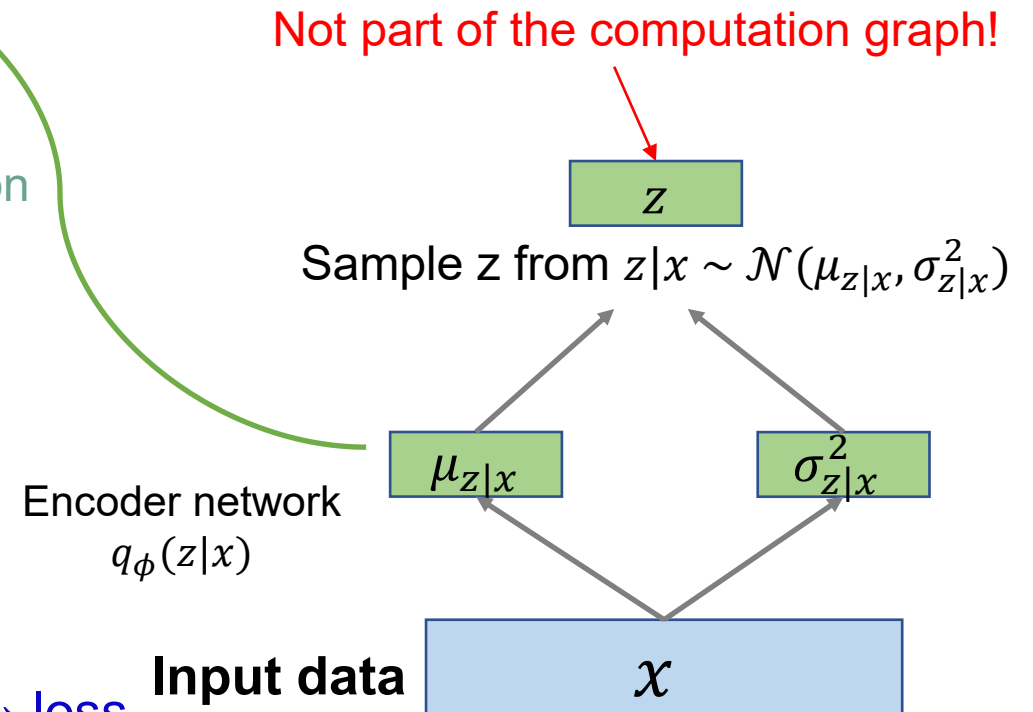


Variational Autoencoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbf{E}_z[\log p_\theta(x^{(i)}|z)] - \mathbf{D}_{KL}(q_\phi(z|x^{(i)})||p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Make approximate posterior distribution close to prior



The chain: $\phi \rightarrow \text{Encoder}\phi(x) \rightarrow \mu, \sigma \rightarrow z \rightarrow \text{Decoder}\theta(z) \rightarrow \text{loss}.$

Variational Autoencoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbb{E}_z[\log p_\theta(x^{(i)}|z)] - \mathbf{D}_{KL}(q_\phi(z|x^{(i)})||p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Make approximate posterior distribution close to prior

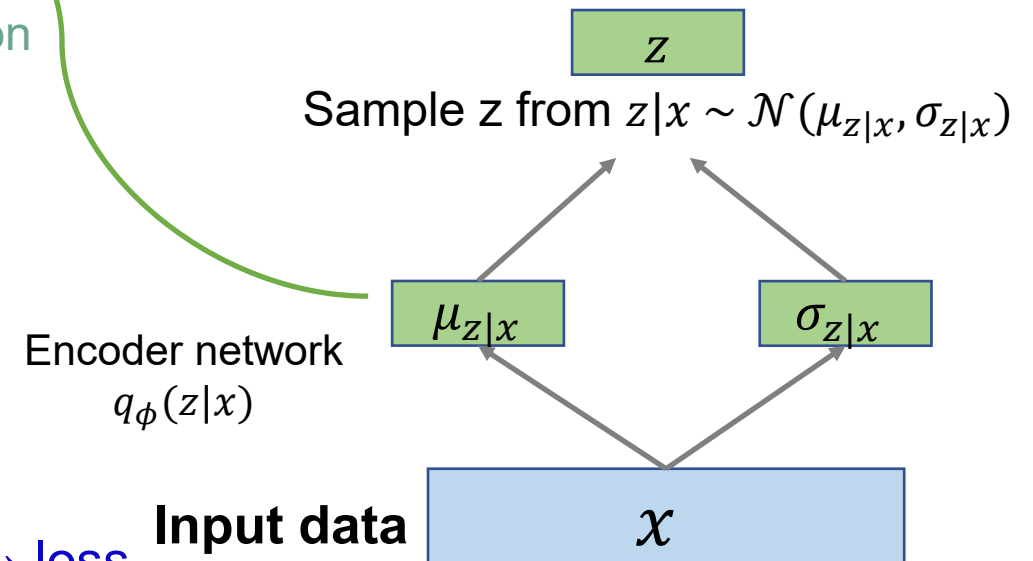
Reparameterization trick to make sampling differentiable:

Sample $\epsilon \sim \mathcal{N}(0, I)$

$$z = \mu_{z|x} + \epsilon \sigma_{z|x}$$

Input to the graph

Part of computation graph



The chain: $\phi \rightarrow \text{Encoder}\phi(x) \rightarrow \mu, \sigma \rightarrow z \rightarrow \text{Decoder}\theta(z) \rightarrow \text{loss}$. **Input data**

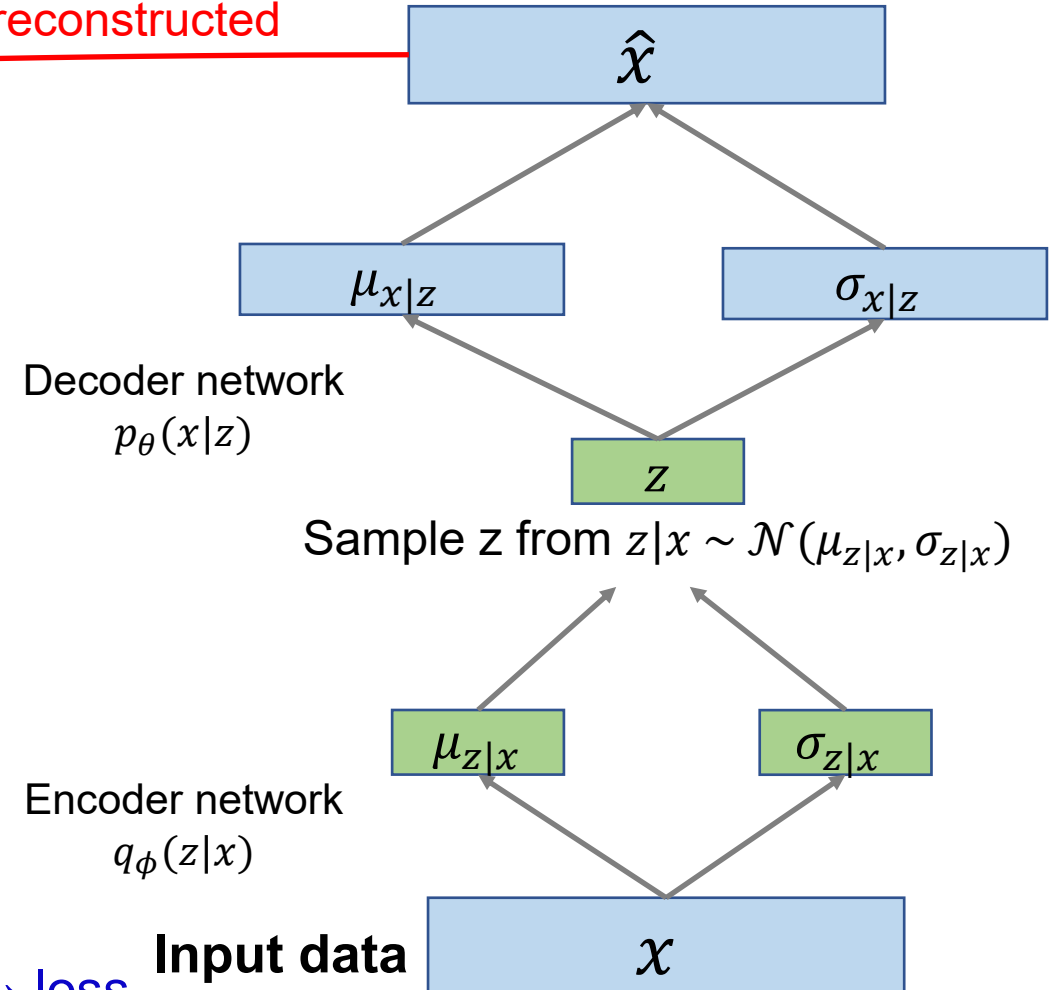
Variational Autoencoders

Putting it all together: maximizing the likelihood lower bound

$$\underbrace{\mathbb{E}_z[\log p_\theta(x^{(i)}|z)] - \mathbf{D}_{KL}(q_\phi(z|x^{(i)})||p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Maximize likelihood of original input being reconstructed

For every minibatch of input data: compute this forward pass, and then backprop!



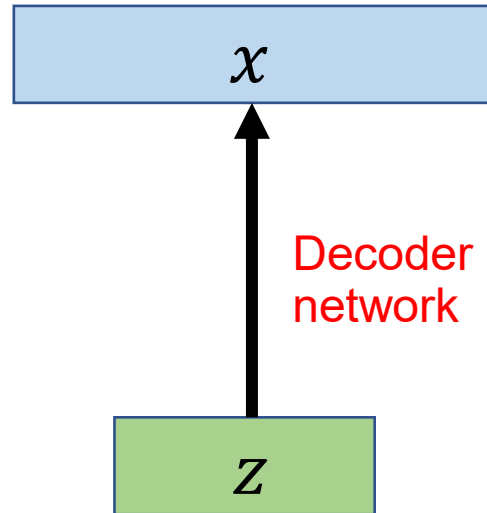
The chain: $\phi \rightarrow \text{Encoder}\phi(x) \rightarrow \mu, \sigma \rightarrow z \rightarrow \text{Decoder}\theta(z) \rightarrow \text{loss}$.

Variational Autoencoders: Generating Data!

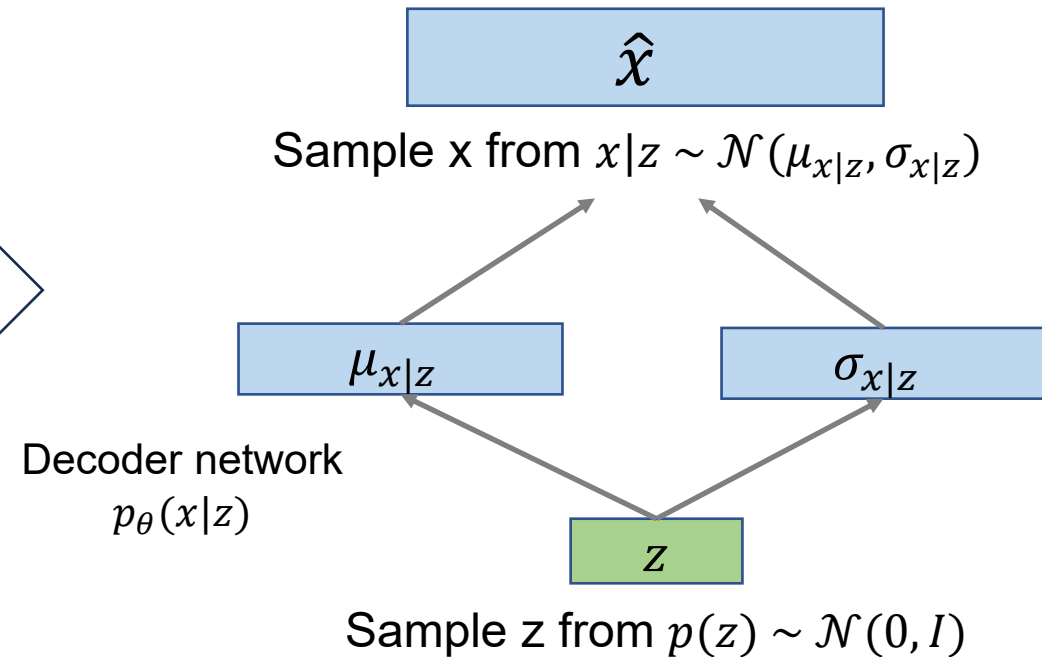
Our assumption about data generation process

Sample from true conditional $p_{\theta^*}(x|z^{(i)})$

Sample from true prior $z^{(i)} \sim p_{\theta^*}(z)$

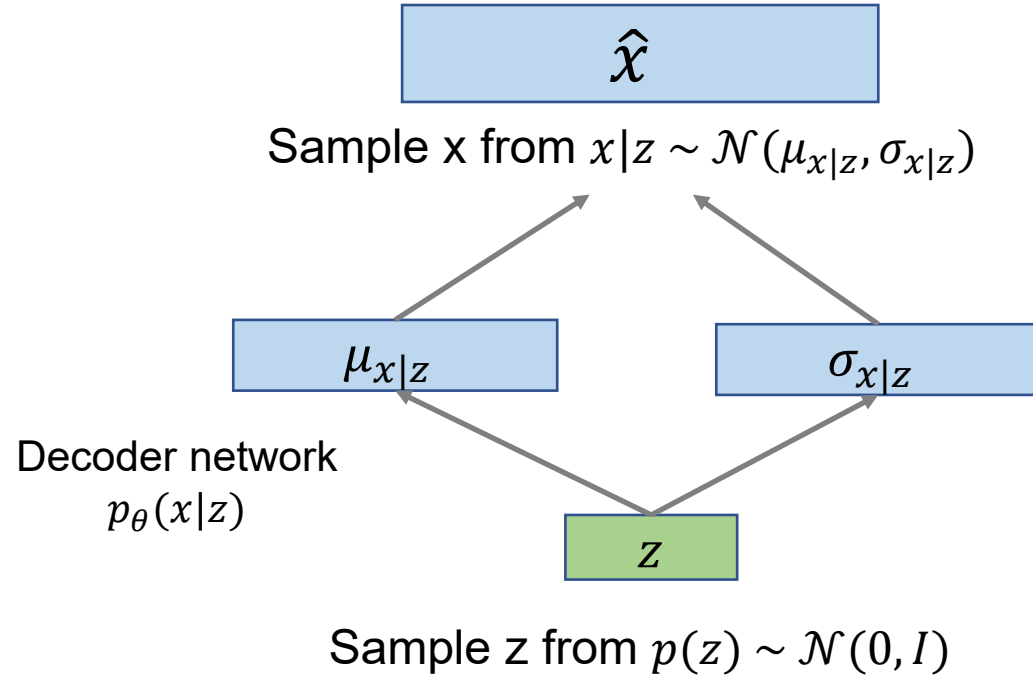


Now given a trained VAE:
use decoder network & sample z from prior!

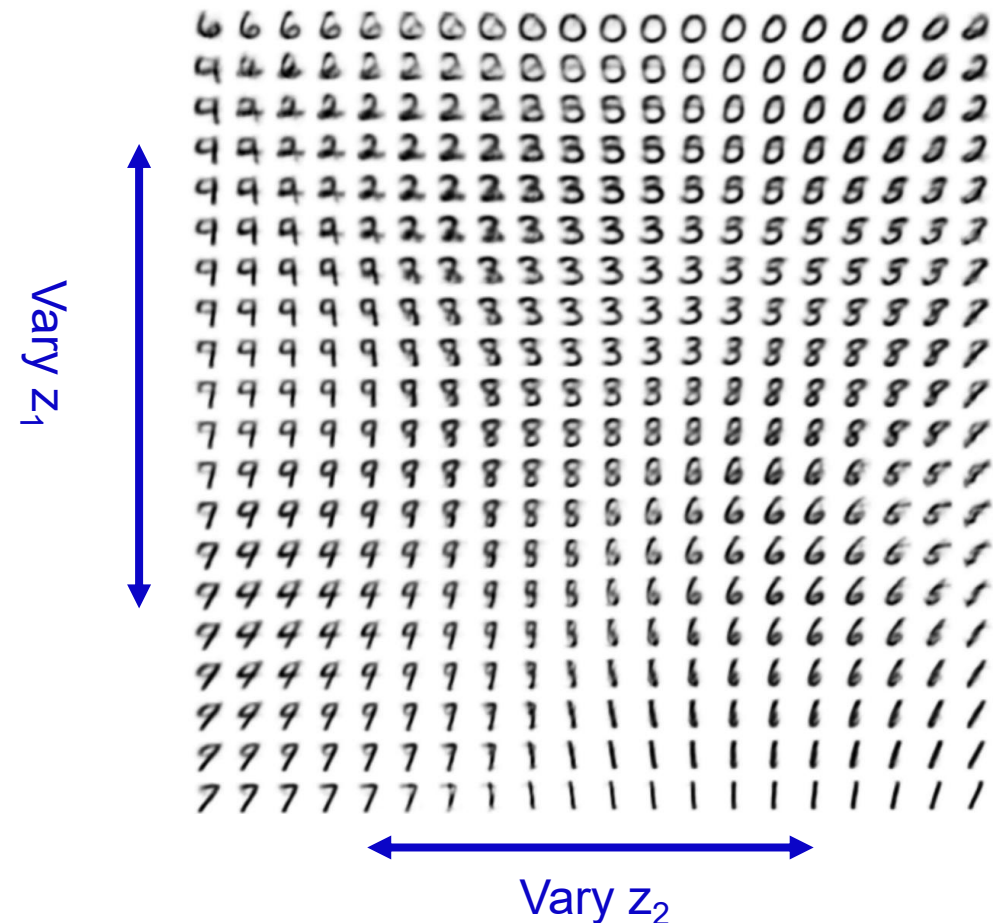


Variational Autoencoders: Generating Data!

Use decoder network. Now sample z from prior!



Data manifold for 2-d z



Variational Autoencoders: Generating Data!

Diagonal prior on $\mathbf{z}$
=> independent
latent variables

Different
dimensions of $\mathbf{z}$
encode
interpretable
factors of variation

Vary z_1
Degree of smile



Vary z_2
Head pose

Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational Autoencoders: Summary

Main idea

- Add probability to autoencoders so we can sample and generate data.
- Exact likelihood is hard, so VAEs optimize the **ELBO**.

Strengths

- Principled probabilistic generative model.
- Smooth, sampleable latent space.
- Encoder $q(z | x)$ gives useful representations.

Limitations

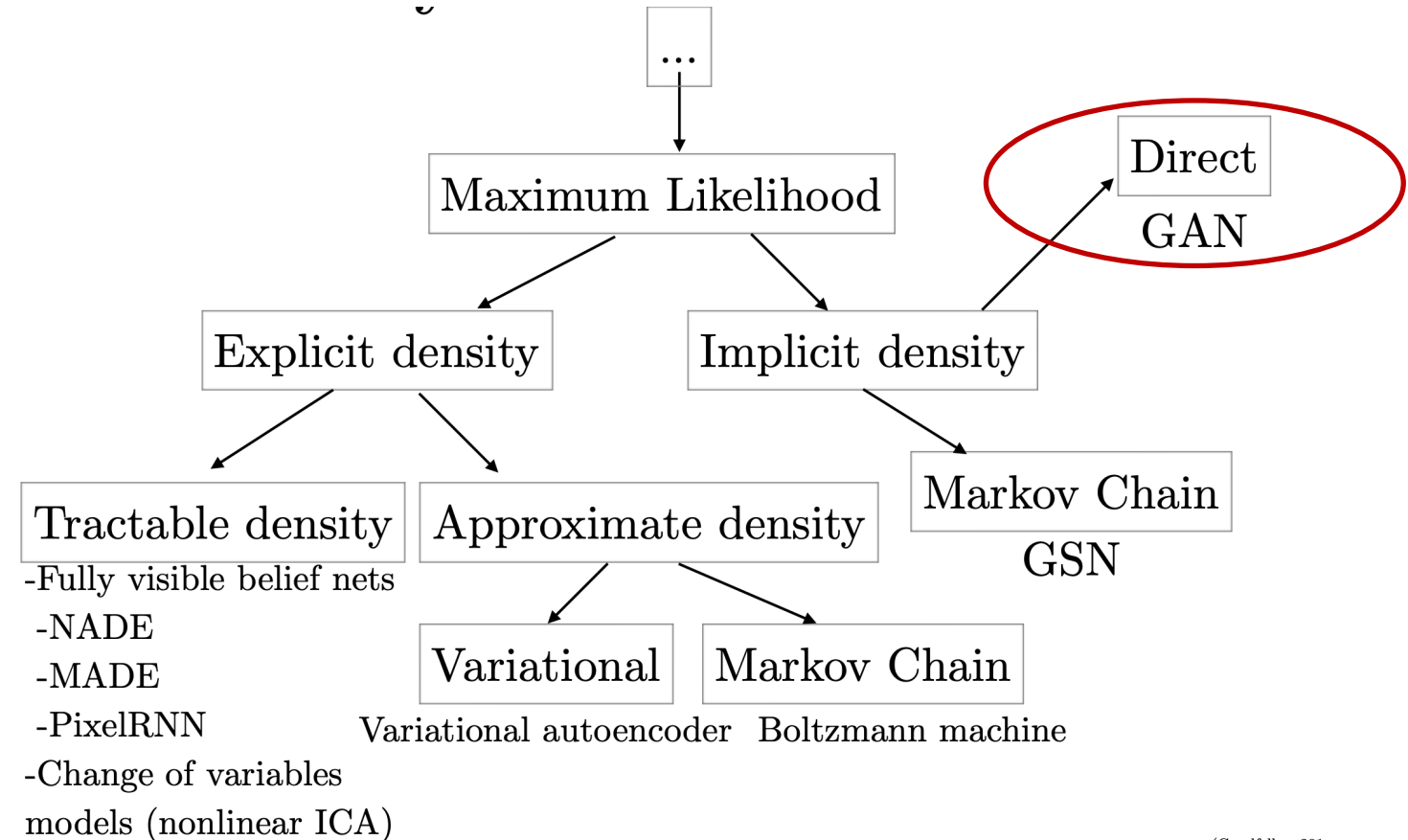
- Optimizes a lower bound, not exact likelihood.
- Samples can be blurrier than GANs/diffusion models.

Research directions

- More flexible posteriors/priors.
- Better disentangled representations.

Outline

- PixelRNN/PixelCNN
- VAE
- GAN
- Diffusion (Week 9)



So far...

PixelRNN/PixelCNN define tractable density function, optimize likelihood of training data:

$$p_{\theta}(x) = \prod_{i=1}^n p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

Variational Autoencoders (VAEs) define intractable density function with latent $\mathbf{z}$:

$$p_{\theta}(x) = \int p_{\theta}(z) p_{\theta}(x|z) dz$$

Cannot optimize directly $\rightarrow$ derive and optimize lower bound on likelihood instead

What if we give up on explicitly modeling density, and just want ability to sample?

GANs: not modeling any explicit density function!

Generative Adversarial Networks

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Problem: Want to sample from complex, high-dimensional training distribution.
No direct way to do this!

Solution: Sample from a simple distribution we can easily sample from, e.g. random noise. Learn transformation to training-data distribution.

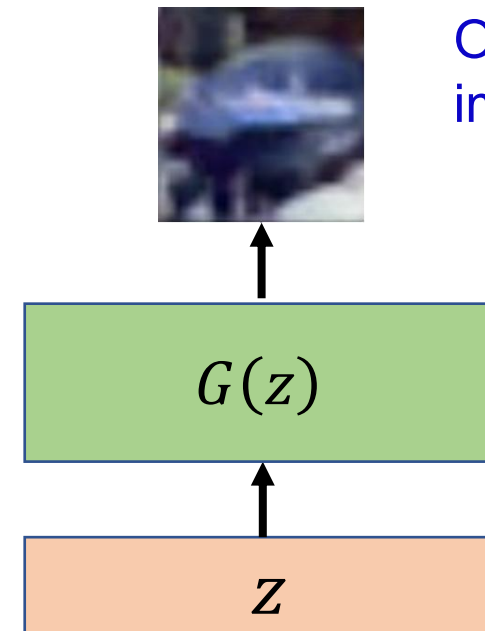
GAN has no encoder and no known correspondence between a random noise vector z and a particular training image x – can't reconstruct specific examples

Output: Sample from training distribution

Objective: generated images should look “real”

Generator Network

Input: Random noise



Generative Adversarial Networks

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Problem: Want to sample from complex, high-dimensional training distribution.
No direct way to do this!

Solution: Sample from a simple distribution we can easily sample from, e.g. random noise. Learn transformation to training-data distribution.

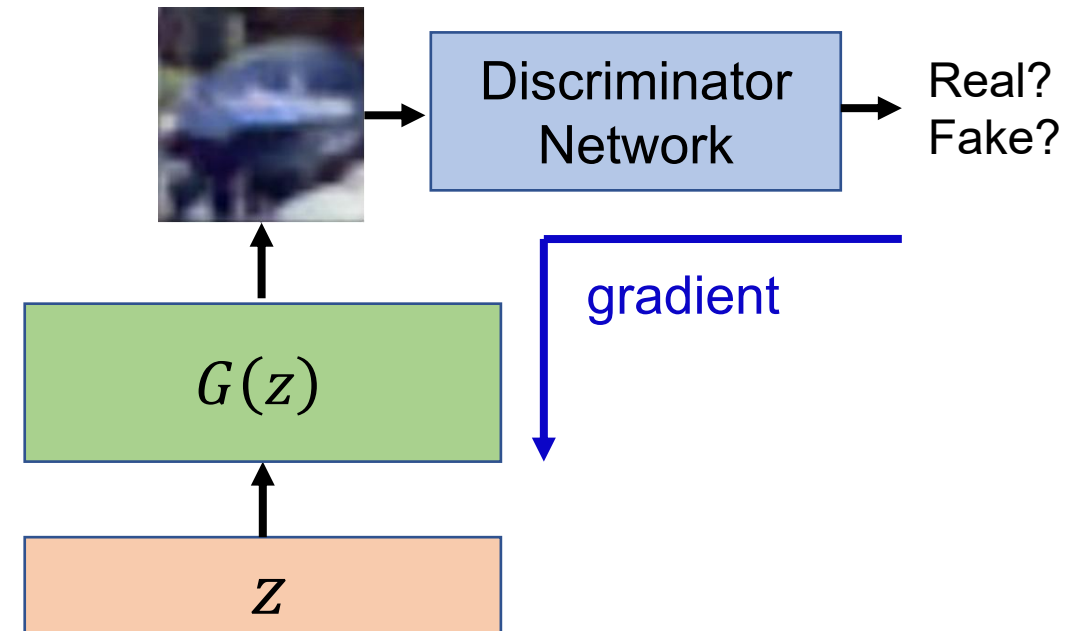
GAN has no encoder and no known correspondence between a random noise vector z and a particular training image x – can’t reconstruct specific examples

Solution: Use a discriminator network to tell whether the generate image is within data distribution (“real”) or not

Output: Sample from training distribution

Generator Network

Input: Random noise

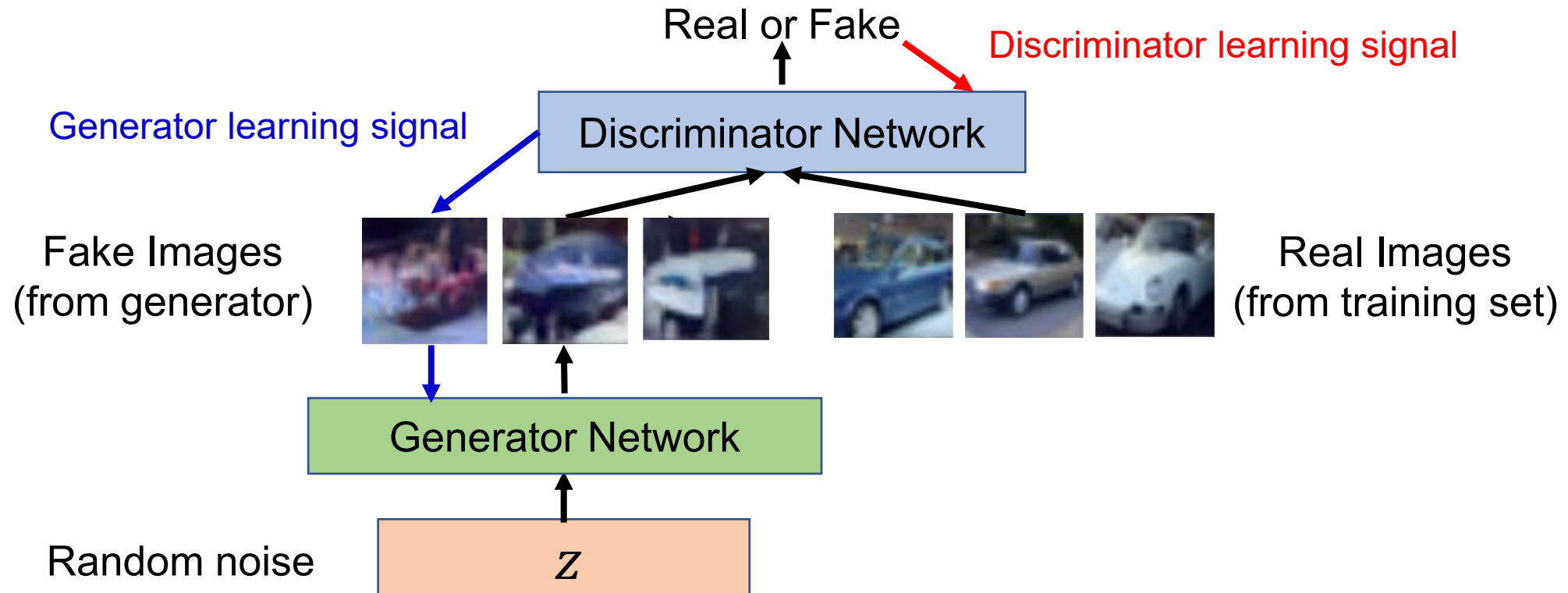


Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images



Training GANs: Two-player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **min-max game**

Min-max objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Generator objective Discriminator objective

Training GANs: Two-player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

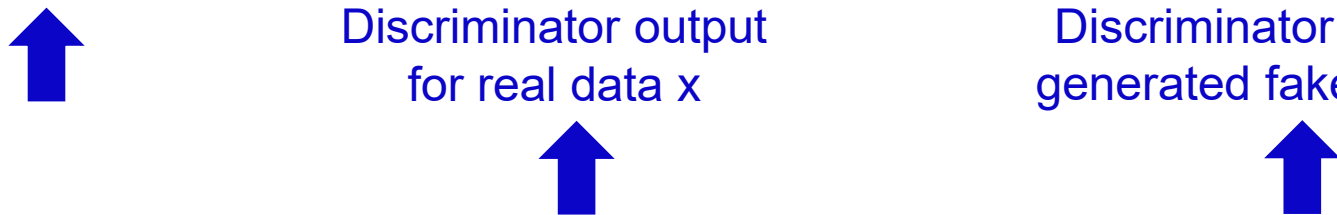
Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **min-max game**

Discriminator outputs likelihood in (0,1) of real image

Min-max objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \underbrace{\log D_{\theta_d}(x)}_{\substack{\text{Discriminator output} \\ \text{for real data } x}} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\substack{\text{Discriminator output for} \\ \text{generated fake data } G(z)}}) \right]$$


Training GANs: Two-player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **min-max game**

Discriminator outputs likelihood in (0,1) of real image

Min-max objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \underbrace{\log D_{\theta_d}(x)}_{\text{Discriminator output for real data } x} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\text{Discriminator output for generated fake data } G(z)}) \right]$$


Training GANs: Two-player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **min-max game**

Discriminator outputs likelihood in (0,1) of real image

Min-max objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \underbrace{\log D_{\theta_d}(x)}_{\substack{\text{Discriminator output} \\ \text{for real data } x}} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\substack{\text{Discriminator output for} \\ \text{generated fake data } G(z)}}) \right]$$

- Discriminator (θ_d) wants to **maximize objective** such that $D(x)$ is close to 1 (real) and $D(G(z))$ is close to 0 (fake)
- Generator (θ_g) wants to **minimize objective** such that $D(G(z))$ is close to 1 (discriminator is fooled into thinking generated $G(z)$ is real)

Training GANs: Two-player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Min-max objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

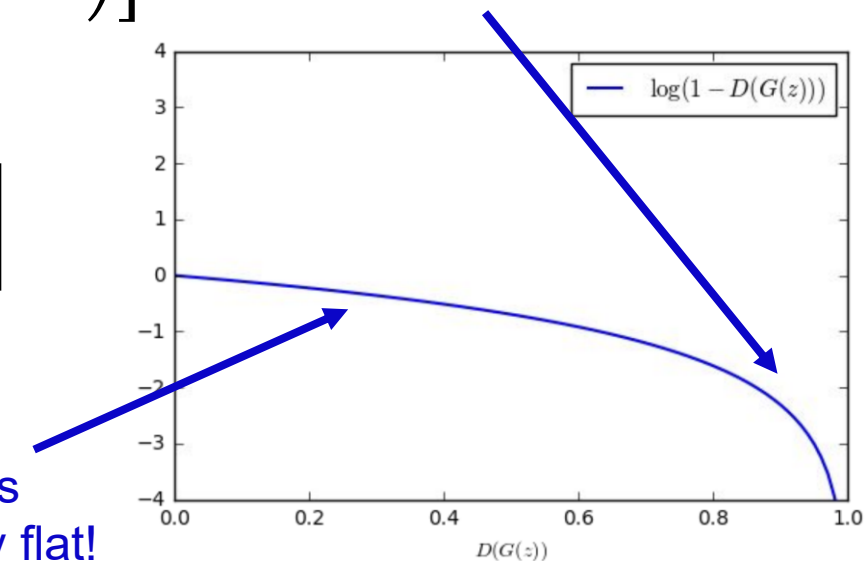
2. **Gradient descent** on generator

$$\min_{\theta_g} \left[\mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

In practice, optimizing this generator objective does not work well!

But gradient in this region is relatively flat!

Gradient signal dominated by region where sample is already good



Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Min-max objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

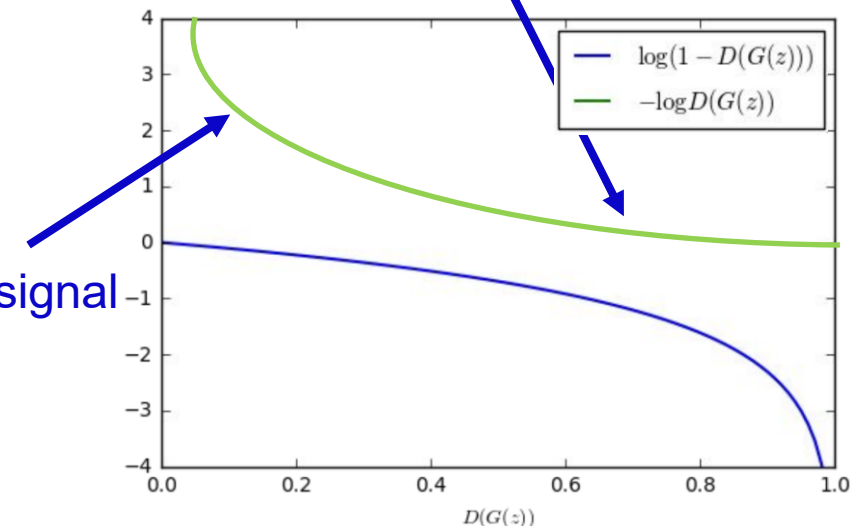
$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Instead: Gradient ascent** on generator, **different objective**

$$\max_{\theta_g} \left[\mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z))) \right]$$

High gradient signal

Low gradient signal



Instead of minimizing likelihood of discriminator being correct, now maximize likelihood of discriminator being wrong.

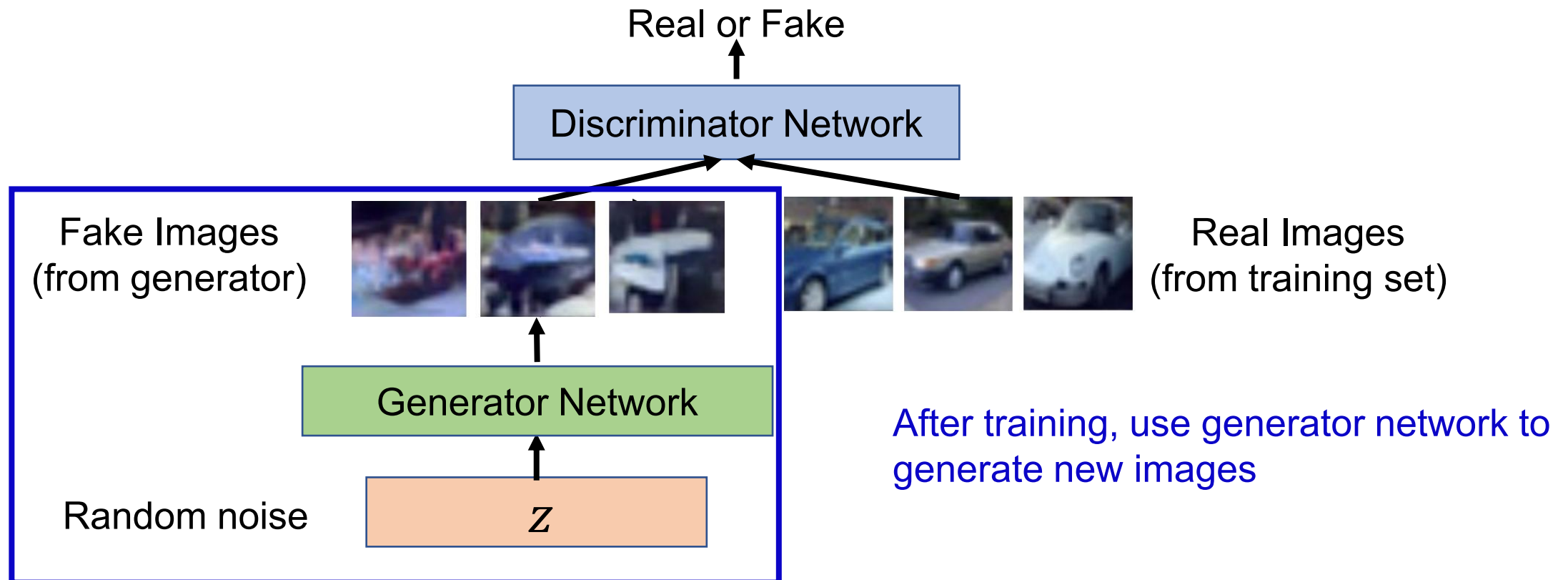
Same objective of fooling discriminator, but now higher gradient signal for bad samples => works much better! Standard in practice

Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images

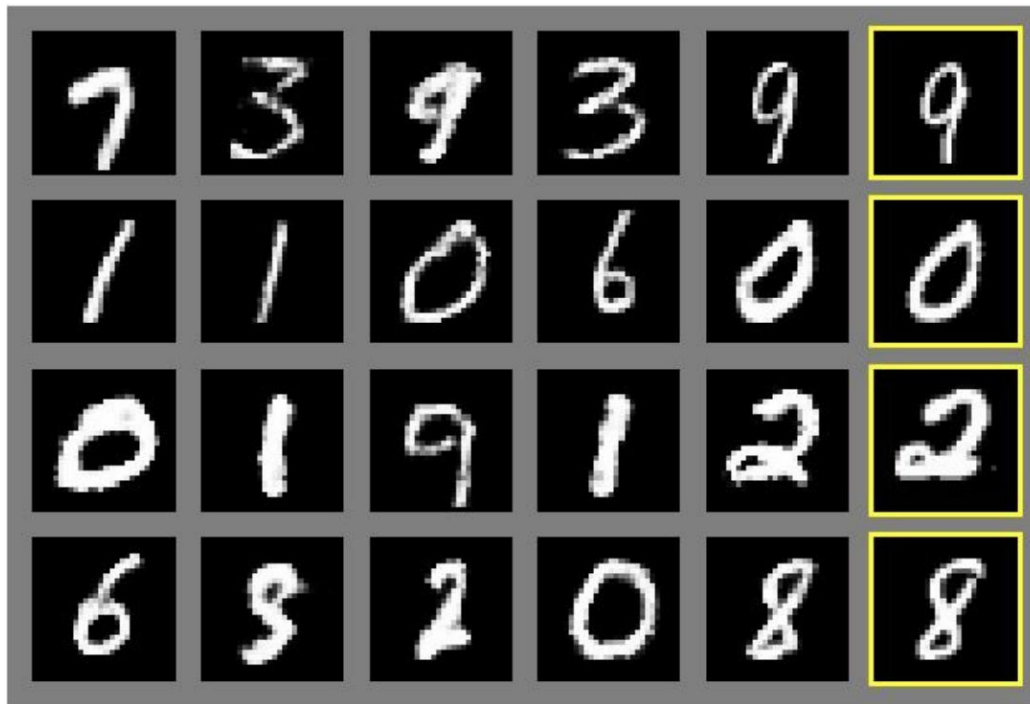
Generator network: try to fool the discriminator by generating real-looking images



Generative Adversarial Nets

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Generated samples



Generated samples

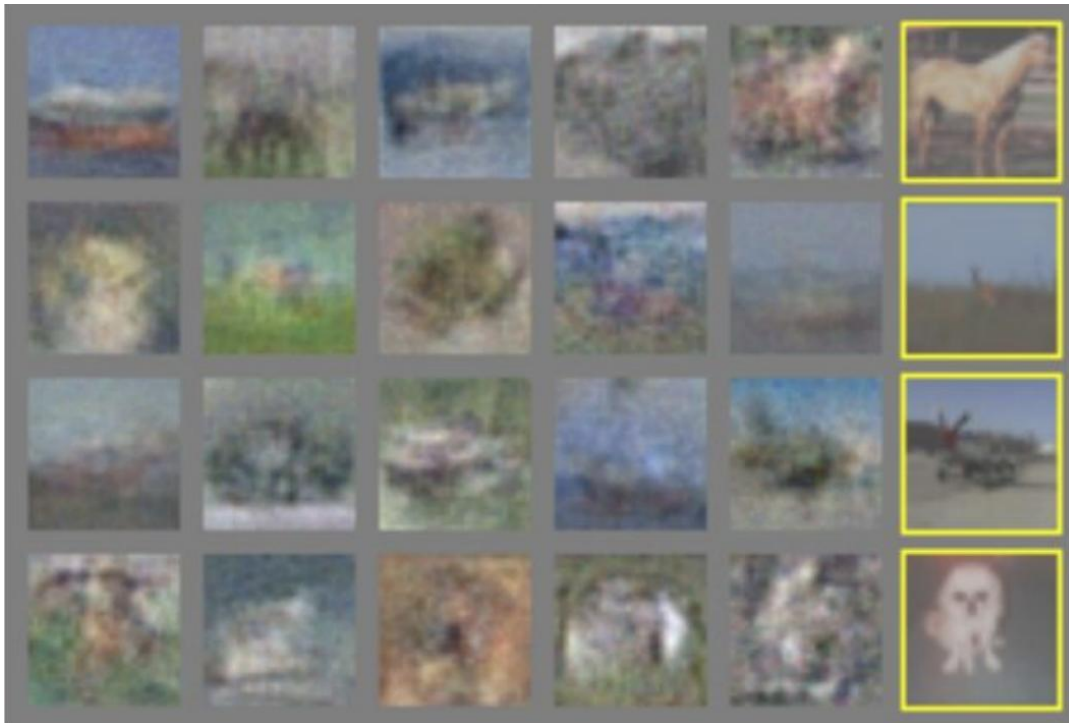


Nearest neighbor from training set

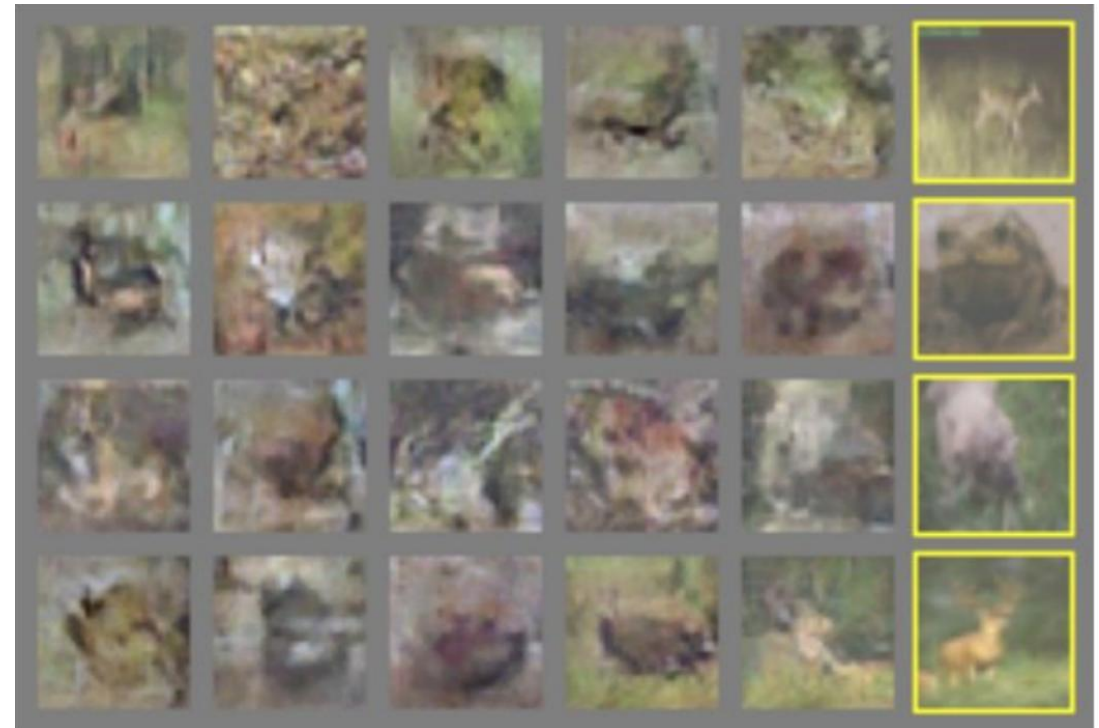
Generative Adversarial Nets

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Generated samples



Generated samples



Nearest neighbor from training set

Generative Adversarial Nets: Interpretable Vector Math

Raford et al., ICLR 2016

Smiling woman

Neutral woman

Neutral man

Samples from the model



Average Z vectors, do arithmetic



Smiling Man



Generative Adversarial Nets: Interpretable Vector Math

Glasses man No glasses man No glasses woman

Raford et al., ICLR 2016



Woman with glasses



Generative Adversarial Nets: Summary

Main idea

- No explicit density $p(x)$.
- Learn generation through a two-player game: **Generator vs. Discriminator**.

Strengths

- Can produce high-quality, realistic samples.

Limitations

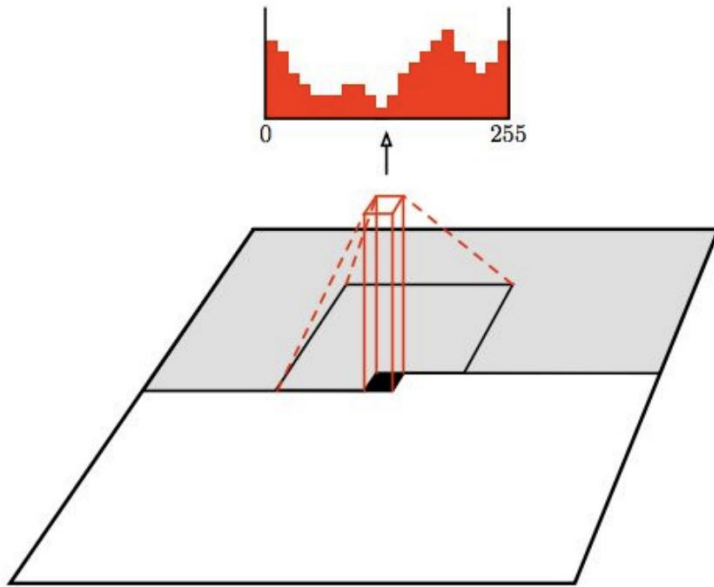
- Training is unstable and sensitive.
- Cannot directly compute $p(x)$ or $p(z | x)$.

Research directions

- More stable objectives: WGAN, LSGAN, etc.
- Conditional GANs and application-specific GANs.

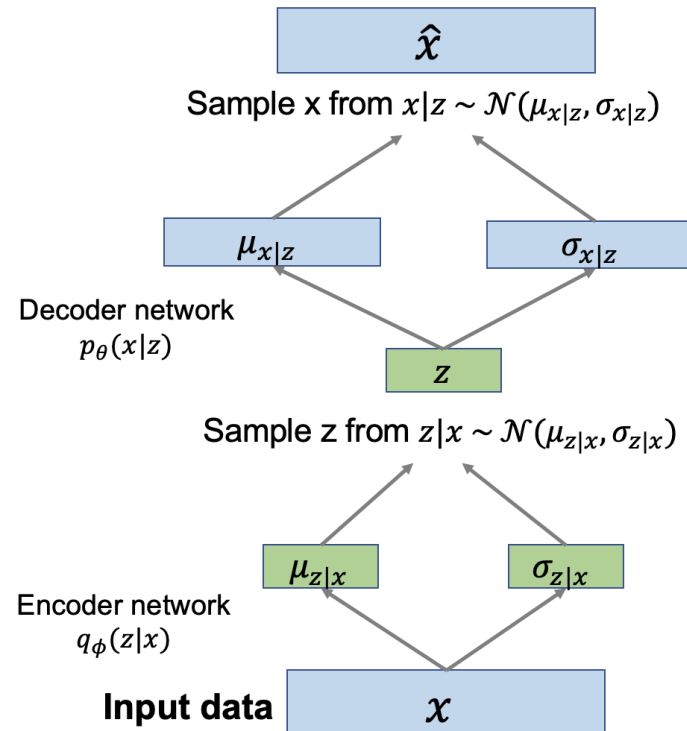
Summary

Autoregressive models: PixelRNN, PixelCNN



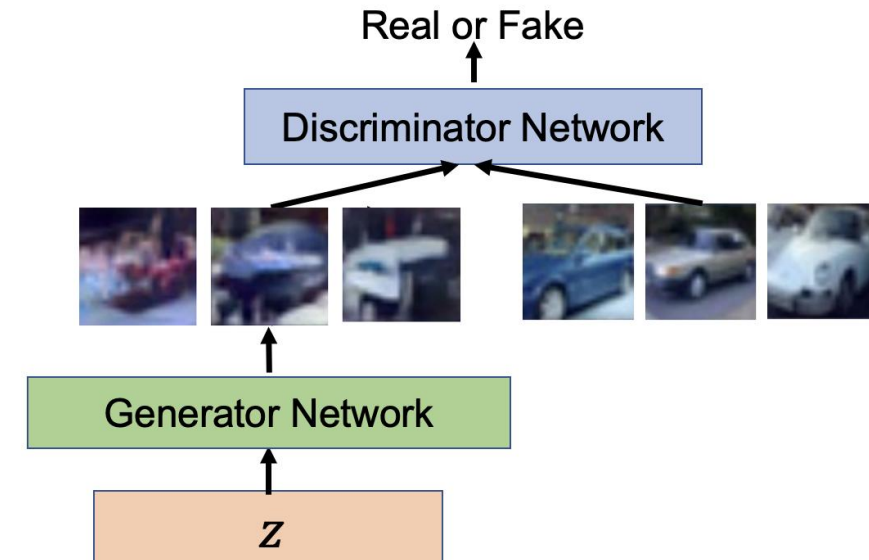
Van der Oord et al, “Conditional image generation with pixelCNN decoders”, NIPS 2016

Variational Autoencoders



Kingma and Welling, “Auto-encoding variational bayes”, ICLR 2013

Generative Adversarial Networks (GANs)



Goodfellow et al, “Generative Adversarial Nets”, NIPS 2014